Nafs-Aql Technical Specification

Version 1.1 (18 October 2026; adds the Phase 7 requirements NFR-030, NFR-031, FR-080, FR-081 and ER-020)

Dirac R&D Team

Abstract

This document specifies Nafs-Aql, a verifier-gated dual-process (System 1 / System 2) chat system built from small Qwen models and deployable on a CPU-only machine, in enough detail for an engineering team to implement it without reading the companion technical report. Normative statements use RFC 2119 keywords; every requirement has a unique identifier (`FR`, `NFR`, `IF`, `MR`, `TR`, `SR`, `OR`, `ER`) and appears in the traceability matrix of Section 13. Parameters that come from the formal results of the report are marked with the result that determines them. Latency, throughput and memory figures are *targets*, not measurements, except where stated as measured on the reference machine. Machine-readable artefacts (`openapi.yaml`, JSON Schemas, sample configurations, Cargo workspace skeleton) accompany this document in the same folder.

Contents

1 Scope, Goals, Non-Goals and Definitions **3**
- 1.1 Scope . . . 3
- 1.2 Goals . . . 3
- 1.3 Non-goals . . . 3
- 1.4 Keywords . . . 3
- 1.5 Glossary . . . 3
- 1.6 Definitions . . . 3

2 Requirements **3**
- 2.1 Functional requirements . . . 4
- 2.2 Non-functional requirements . . . 7
- 2.3 Interface requirements . . . 8
- 2.4 Model requirements . . . 9
- 2.5 Training requirements . . . 10
- 2.6 Security, safety and ethics requirements . . . 10
- 2.7 Operations requirements . . . 11
- 2.8 Evaluation and test requirements . . . 12

3 System Architecture **12**
- 3.1 Components and responsibilities . . . 12
- 3.2 Data flow . . . 13
- 3.3 Deployment topology . . . 13

4 Component Specifications **14**
- 4.1 System 1 (fast path) . . . 14
- 4.2 System 2 (slow path) . . . 14
- 4.3 Aql Gate (controller) . . . 14
- 4.4 Verifier Hub and arbiter . . . 15
- 4.5 Context Manager . . . 16
- 4.6 Muhasaba Ledger . . . 16

4.7 Rust Gateway . . . 16
4.8 Web UI and Eval Harness . . . 16
4.9 Derived parameters . . . 16

5 Model Specification 16
5.1 Selected models . . . 16
5.2 Thinking control (MR-002) . . . 17
5.3 vLLM launch configuration (Linux profile) . . . 17
5.4 llama-server configuration (macOS and AVX2 hosts) . . . 18
5.5 Fallbacks . . . 18

6 Training and Fine-Tuning Specification 18
6.1 Data schema (TR-001) . . . 18
6.2 Dataset construction (FR-070, TR-002, TR-008) . . . 18
6.3 Synthetic data (TR-003) . . . 18
6.4 Unsloth configuration (TR-004, TR-005) . . . 18
6.5 Stages and acceptance (TR-006, TR-007) . . . 19

7 Interface Specification 19
7.1 External API (IF-001–IF-004, IF-009) . . . 19
7.2 Internal contracts (IF-005, IF-010) . . . 19
7.3 Backend integration (IF-006–IF-008) . . . 19

8 Rust Implementation Specification 20
8.1 Workspace . . . 20
8.2 Crates and rationale . . . 20
8.3 Module boundaries and key types . . . 20
8.4 Concurrency model . . . 21
8.5 Configuration . . . 21
8.6 Logging and tracing . . . 21
8.7 Error taxonomy . . . 21

9 Web UI Specification 21
9.1 User stories . . . 21
9.2 Screens . . . 21

10 Evaluation and Test Specification 21
10.1 Metrics . . . 21
10.2 Benchmarks and licences . . . 22
10.3 Experiment protocol and statistics . . . 22
10.4 Acceptance thresholds (targets) . . . 23
10.5 Test plans . . . 23

11 Security, Safety and Ethics 23
11.1 Threat model . . . 23
11.2 Controls . . . 23

12 Operations 24

13 Traceability Matrix 24

14 Open Issues and Risks 27

1 Scope, Goals, Non-Goals and Definitions

1.1 Scope

This specification covers the gateway, orchestrator, controller, verifier hub, context manager, self-model ledger, evaluation harness, performance console, web UI, model serving configuration, and the fine-tuning pipeline of Nafs-Aql. It does not cover model pre-training, the internals of vLLM, llama.cpp or Unsloth, or any multimodal capability.

1.2 Goals

G1. Answer chat requests with a fast path by default and a deliberate, verified path when the expected value of deliberation exceeds its cost.
G2. Attach an honest assurance level to every reply and never present unverified agreement as verification.
G3. Run end to end on a single CPU-only machine of the reference class (four cores, AVX2, 32 GB) with bounded latency, and scale out to a GPU without changing the application.
G4. Improve the fast path over time from verified deliberation traces under a statistically sized promotion gate.
G5. Measure itself: routing quality, calibration, cost, verifier error rates and drift, through an evaluation harness and dashboard.

1.3 Non-goals

Training a base model; autonomous tool use beyond the verifiers; multimodal input; claims of consciousness, understanding or religious authority; interactive slow-path latency on the CPU baseline.

1.4 Keywords

The key words MUST, MUST NOT, SHOULD, SHOULD NOT and MAY are to be interpreted as in RFC 2119. Statements without a key word are informative.

1.5 Glossary

Table 1 maps each borrowed term to its engineering meaning. **Every entry is an analogy**; the code uses the engineering names only, and no claim about the metaphysics of the terms is made.

1.6 Definitions

Fast path (System 1) a model call in non-thinking mode producing a draft and k cheap samples.
Slow path (System 2) a model call in thinking mode with a thinking budget b, sampled sequentially up to N candidates per round for up to R rounds.
Verifier a procedure returning pass/fail with published true-accept rate r and false-accept rate φ. *Exact* verifiers have $\varphi = 0$ on their domain.
Assurance level one of `verified`, `consensus`, `unverified`, `abstained`.
Decision record the gate's inputs, estimates, threshold and outcome for one request.
Trace the record of one request's path through the automaton.
Profile a named pair of model backends (A reference, B minimal, C upgrade).
Reference machine Intel Core i3-8100 (4 cores, AVX2, no AVX-512), 32 GB DDR4-2400, macOS 15.4.1.

2 Requirements

Requirements are generated from `data/requirements.csv`, the single source of truth shared with the roadmap. Priorities use MoSCoW. The phase column refers to the roadmap phase that satisfies

Table 1: Glossary (analogy mapping)

Term (source)	Classical sense (summary)	Engineering component (analogy)
nafs (Qur'an 12:53, 75:2, 89:27; al-Ghazali)	the self; in its lower sense the seat of impulse; described in states: commanding, self-reproaching, at rest	the request's trace state: A unverified draft, L flagged, T accepted; plain engineering names M (retry) and X (abstain) for the rest
'aql (al-Ghazali: the vizier)	the deliberating intellect that counsels the heart	Aql Gate: cost–risk arbitration between fast and slow paths; System 2 deliberation
qalb (al-Ghazali: the king)	the integrating executive that consults the intellect and commands the faculties	orchestrator `nafs-core`: owns the request lifecycle
wahm, *dhakira* (Ibn Sina)	estimation (quick particular judgement), memory	gate features (cheap uncertainty signals); Context Manager
muraqaba, *muhasaba* (al-Muhasibi, al-Ghazali)	vigilance and self-accounting under a standard	Muhasaba Ledger: audit stream, calibration tables, drift monitor
tazkiyat al-nafs	purification: gradual change of disposition	Tazkiya Distillation: verified traces to LoRA, promotion gate
ruh, *ilham*, Active Intellect	spirit; inspiration; separate intellect	**not mapped** (see report, Section II-A)

the requirement.

2.1 Functional requirements

Table 2: Functional requirements

ID	Title	Normative statement	Priority	Phase
FR-001	Chat rooms	The gateway MUST provide multi-user chat rooms with streamed assistant replies.	Must	P1
FR-002	Reply metadata	Every reply MUST carry reasoning_mode (fast or slow), assurance level, tokens used, thinking budget used and wall-clock cost.	Must	P1
FR-003	Conversation context	The Context Manager MUST assemble prompts from conversation history within the configured context cap and SHOULD preserve prompt-cache prefixes.	Must	P1
FR-004	Speed-care control	Users MAY set a single scalar lambda (speed-care) per conversation; the gate MUST use it as the cost price.	Should	P2
FR-005	Model profiles	Operators MUST be able to select model profile A, B or C by configuration without code changes.	Must	P1
FR-006	Controller events	The gateway MUST stream controller events (gate decision, verification result, retry, abstention) alongside tokens.	Must	P1
FR-010	Gate features	The Aql Gate MUST compute, from the fast draft, the features: mean and minimum token log-probability, agreement among k samples, domain class and, for verifiable tasks, the verifier outcome.	Must	P2

ID	Title	Normative statement	Priority	Phase
FR-011	Derived gate rule	The gate MUST escalate iff L*(e1_hat - e2_hat) > lambda*delta_c using a calibrated estimator e1_hat and per-domain e2_hat; hand-tuned score thresholds MUST NOT be used in production mode.	Must	P2
FR-012	Failed verification escalates	A verifiable task whose fast draft fails verification MUST be escalated and MUST NOT be returned as kept.	Must	P1
FR-013	Decision record	For every request the gate MUST write a decision record containing features, estimates, threshold, lambda, configuration hash and outcome.	Must	P1
FR-014	Budget mode	Operators MAY configure an escalation budget B instead of lambda; the gate MUST then use the multiplier lambda* from the empirical quantile with the DKW sample-size rule.	Could	P2
FR-015	Gate recalibration	The calibration estimator MUST be retrained from the ledger on a configured schedule and MUST NOT be promoted unless NFR-020 holds on held-out data.	Must	P2
FR-020	Sequential verified sampling	System 2 MUST draw candidates sequentially, verify each, stop at the first accepted candidate, and respect caps N per round and R rounds.	Must	P2
FR-021	Caps from Phase-0 fit	N and R per domain MUST be derived from the Phase-0 Beta fit of acceptance rates and the configured cost ratio c_p/L_a, and the derivation MUST be recorded with the configuration.	Must	P2
FR-022	No unverified refinement	On non-verifiable tasks R MUST be at most 1 and no refinement MAY replace an earlier candidate without a verifier or judge acceptance.	Must	P2
FR-023	Draft-blind deliberation	System 2 MUST NOT receive the fast draft by default; draft-conditioned mode MAY exist only as an ablation flag that is off in production.	Must	P1
FR-024	Thinking budget and forcing	Each domain MUST have a thinking budget; on exhaustion the controller MUST force the answer by closing the reasoning span and, if no answer results, return an abstention with notice.	Must	P1
FR-025	Evidence-fed retry	A rejected candidate's verifier evidence MUST be included in the next round's prompt.	Must	P2
FR-026	Bounded loop	The orchestrator MUST enforce the model-call cap 1+N(R+1) and a wall-clock cap per request, terminating in state T or X.	Must	P1
FR-030	Exact verifiers	The Verifier Hub MUST provide exact verifiers for code (test execution in a sandbox), mathematics (numeric and symbolic equality against a reference) and JSON Schema validity.	Must	P1
FR-031	Assurance levels	Every reply MUST carry one of the assurance levels verified, consensus, unverified or abstained, and consensus MUST NOT be presented as verified.	Must	P1

ID	Title	Normative statement	Priority	Phase
FR-032	Verifier rates	Each verifier MUST publish (r, phi) measured by mutation testing; exact verifiers MUST use acceptance threshold 1.0.	Must	P2
FR-033	Consensus not training data	Consensus-assured outputs MUST NOT enter the distillation set without human review.	Must	P3
FR-034	Judge hard gate	The rubric judge MUST apply hard floors honesty ≥ 0.5, harm-avoidance ≥ 0.5, helpfulness ≥ 0.5 and total V ≥ 0.7 before any candidate is accepted on non-verifiable tasks.	Should	P2
FR-035	Code sandbox	Code verification MUST run in a subprocess with CPU-time, memory, wall-clock and output limits, no network and an ephemeral file system.	Must	P1
FR-040	Audit stream	A random fraction epsilon (default 0.05) of kept requests MUST also be run through System 2 offline and recorded as audits.	Must	P2
FR-041	Capability table	The Muhasaba Ledger MUST produce per-domain capability statistics keyed by model and adapter hash and MUST report domains without audited data as unmeasured.	Must	P2
FR-042	Drift monitoring	The ledger MUST run a CUSUM chart on calibration error and accuracy per domain with a negative control and raise an alert on change.	Should	P4
FR-050	Evaluation harness	The Eval Harness MUST run benchmark sets through the gateway API for the system and for the fast-only and slow-only baselines and compute the metrics of ER-010.	Must	P0
FR-051	Paired statistics	The harness MUST report paired differences with confidence intervals, non-inferiority tests and Bonferroni correction across domains.	Must	P4
FR-052	Performance console	The performance console MUST run load tests against the gateway and report latency quantiles, throughput, escalation rate and cost per request.	Must	P4
FR-053	Gate dashboard	The dashboard MUST display the go/no-go status of each roadmap gate from stored evaluation runs.	Should	P4
FR-060	Persistent store	Conversations, decision records and traces MUST be stored (SQLite and JSONL) with a configurable retention period.	Must	P1
FR-061	Retrieval memory	The Context Manager MAY retrieve prior verified answers with an embedding index; retrieved text MUST be delimited as data with provenance.	Could	P5
FR-070	Distillation dataset builder	The dataset builder MUST select verified System 2 traces, deduplicate, cap any domain at 30 percent, and add 5 percent abstention examples.	Must	P3

ID	Title	Normative statement	Priority	Phase
FR-080	Early escalation	Under the fixed rule the gate SHOULD stop an unverifiable draft once its running mean log-prob after a minimum number of tokens is below the threshold minus a margin, and escalate; its decisions MUST agree with full-draft decisions on at least 95% of a held-out prompt set before it is on by default.	Should	P7
FR-081	Speculative decoding heads	Each model MAY be served with a trained speculative head (EAGLE-3) or an n-gram drafter, enabled only where a measured decode speed-up of at least 1.2x holds with identical greedy output.	Could	P7
FR-082	Adaptive reasoning exit	System 2 reasoning SHOULD be probed at reflection points (and at least every configured interval) for a short answer; on verifiable tasks the exact verifier MAY end the reasoning early, and on other tasks agreement between consecutive probes MAY; the final answer MUST be generated after the reasoning as on the standard path, and the exit reason MUST be recorded.	Should	P7

2.2 Non-functional requirements

Table 3: Non-functional requirements

ID	Title	Normative statement	Priority	Phase
NFR-001	Hard latency bound	Every request MUST terminate within the configured wall-clock cap (default 300 s on the CPU baseline), returning an abstention if necessary. Target.	Must	P1
NFR-002	Fast-path first token	Target: on the reference machine, time to first token of the fast path SHOULD be $\leq$ 2 s at p95 for prompts $\leq$ 1000 tokens with prompt cache hits.	Should	P1
NFR-003	Fast-path reply latency	Target: a 150-token fast-path reply SHOULD complete within 12 s at p95 on the reference machine (Profile A).	Should	P1
NFR-004	Slow-path latency	Target: a slow-path reply with a 1024-token thinking budget on the 4B model SHOULD complete within 240 s at p95 on the reference machine.	Should	P1
NFR-005	Memory budget	Resident memory of both model servers, KV cache and the gateway MUST stay below 8 GB in Profile A on the CPU baseline.	Must	P1
NFR-006	Context cap	The CPU baseline MUST cap prompts at 4096 tokens and MUST enable prompt caching on the model servers.	Must	P1

ID	Title	Normative statement	Priority	Phase
NFR-007	Concurrency	Target: the reference machine SHOULD serve 4 concurrent chat sessions with p95 fast-path latency at most twice the single-session value.	Could	P4
NFR-010	Backend resilience	The gateway MUST survive model-server restarts, reconnect automatically and lose no decision records or traces.	Must	P2
NFR-011	Backpressure	In-flight requests per backend MUST be bounded by configuration and excess requests MUST receive HTTP 429 with a retry hint.	Must	P1
NFR-012	Pinned model identity	Model identifiers, revisions, quantisation, chat-template hash and configuration hash MUST be recorded in every trace.	Must	P0
NFR-020	Gate calibration	Target: the calibration error (ECE with 5 bins on $\geq$ 2900 held-out samples) of e1_hat MUST be $\leq$ 0.05 before a calibration is promoted.	Must	P2
NFR-021	Observability	The gateway MUST expose Prometheus metrics, structured JSON logs and OpenTelemetry spans carrying the request identifier.	Must	P1
NFR-022	Access control	API access MUST require a bearer token; rate limits MUST apply per token; secrets MUST NOT appear in logs or traces.	Must	P1
NFR-023	Build reproducibility	Cargo.lock MUST be committed, CI MUST build and test on Linux and macOS, and evaluation runs MUST use recorded random seeds.	Must	P0
NFR-030	Latency attribution	Every reply MUST record per-stage timings (retrieval, draft, samples, verification, deliberation, judge) and, for model stages, the queue wait, prefill and decode time and KV-cache reuse reported by the server; the gateway MUST report each stage's share of wall time.	Must	P7
NFR-031	Interleaved tuning evidence	A serving-flag or weight-format change MUST be justified by an interleaved A/B (alternating order, at least two rounds) on the reference machine, with the host load recorded.	Must	P7

2.3 Interface requirements

Table 4: Interface requirements

ID	Title	Normative statement	Priority	Phase
IF-001	OpenAPI document	The gateway MUST publish an OpenAPI 3.1 document equal to spec/openapi.yaml at /openapi.json.	Must	P1
IF-002	Chat endpoints	The gateway MUST provide POST /v1/chat for complete replies and POST /v1/chat/stream for SSE with event types token, decision, verification, retry, done and error.	Must	P1

ID	Title	Normative statement	Priority	Phase
IF-003	Reasoning mode in replies	The reply schema MUST include reasoning_mode, assurance, usage and decision_id fields.	Must	P1
IF-004	Chat-room WebSocket	Chat rooms MUST be served over WebSocket at /ws/rooms/room_id with JSON frames defined in the schemas.	Must	P1
IF-005	JSON Schemas	ChatRequest, ChatReply, DecisionRecord, TraceRecord and VerifierResult MUST be defined as JSON Schema (draft 2020-12) files and generated from the Rust types.	Must	P1
IF-006	Backend contract	Model backends MUST be reached through the OpenAI-compatible /v1/chat/completions API with logprobs, n and chat_template_kwargs, and reasoning content MUST be separated by the server parser.	Must	P1
IF-007	Retry policy	Backend calls MUST be retried at most 3 times with exponential backoff and jitter on connection errors and HTTP 5xx, never on HTTP 4xx, and only when the request is idempotent.	Must	P1
IF-008	Backend backpressure	Each backend MUST have a concurrency semaphore sized to the server's maximum sequences, and queued requests MUST time out with a distinct error.	Must	P1
IF-009	Bench and eval endpoints	The gateway MUST provide /v1/bench/runs and /v1/eval/runs resources to start, list and fetch runs.	Should	P4
IF-010	Telemetry format	Traces MUST be newline-delimited JSON records conforming to TraceRecord with a schema_version field.	Must	P1

2.4 Model requirements

Table 5: Model requirements

ID	Title	Normative statement	Priority	Phase
MR-001	Pinned models	Profile A MUST pin Qwen/Qwen3-1.7B (System 1) and Qwen/Qwen3-4B-Thinking-2507 (System 2) by repository revision and GGUF file hash.	Must	P0
MR-002	Thinking toggle	The backend layer MUST set thinking on or off per request through chat_template_kwargs or model selection, and MUST reject replies whose reasoning content is not separated.	Must	P1
MR-003	Serving configurations	The specification MUST provide a vLLM-CPU launch configuration for Linux and a llama-server command for macOS or AVX2 hosts exposing the same API.	Must	P0

ID	Title	Normative statement	Priority	Phase
MR-004	Degraded mode	If the System 2 backend is unavailable the gateway MUST answer with the fast path, label the reply unverified, and emit a degraded-mode event.	Must	P2
MR-005	Profile C adoption	Profile C (Qwen3.5) MUST NOT become the default unless hypothesis H7 (decode speed $\geq$ 70 percent of dense Qwen3 at equal size on the target CPU) passes at gate G0 or G5.	Must	P0

2.5 Training requirements

Table 6: Training requirements

ID	Title	Normative statement	Priority	Phase
TR-001	SFT data schema	Training examples MUST follow the JSONL schema with fields id, domain, prompt, answer, rationale ($\leq$ 150 tokens), assurance, verifier, source_trace, adapter_hash.	Must	P3
TR-002	Verified-only training data	Only traces with assurance verified, plus human-reviewed consensus traces, MAY enter the training set.	Must	P3
TR-003	Synthetic data	Synthetic examples MUST be generated by System 2 on benchmark-like prompts, verified by an exact verifier never used for evaluation, and labelled synthetic.	Should	P3
TR-004	Unsloth configuration	Fine-tuning MUST use Unsloth LoRA with rank 16, alpha 16, learning rate 2e-4, 1 to 3 epochs, effective batch 16, dropout 0 on all attention and MLP projections; QLoRA for Qwen3 checkpoints and BF16 LoRA for Qwen3.5 checkpoints.	Must	P3
TR-005	GPU path	Training MUST run on a GPU host (minimum 16 GB VRAM); the CPU machine MUST NOT be used for training.	Must	P3
TR-006	Checkpoints and export	Each training run MUST produce an adapter with a content hash, a merged GGUF in q4_k_m and q8_0, and a manifest linking data snapshot, configuration and hashes.	Must	P3
TR-007	Promotion gate	An adapter MAY be promoted only if the lower confidence bound of the overall gain on a held-out verifier is above zero with point estimate $\geq$ 2 points, no domain's lower bound is below -1 point, NFR-020 and the coupling test still pass, and a human signs off.	Must	P3
TR-008	Replay and abstention	Training sets MUST include 20 percent replay of general instruction data and 5 percent abstention examples.	Should	P3

2.6 Security, safety and ethics requirements

Table 7: Security, safety and ethics requirements

ID	Title	Normative statement	Priority	Phase
SR-001	Prompt-injection handling	Retrieved content, tool output and verifier evidence MUST be delimited as data in prompts; nothing in model output or observed content MAY change controller parameters or configuration.	Must	P2
SR-002	Sandbox isolation	The code sandbox MUST run without network access, under resource limits and, on Linux, under seccomp or a container; sandbox escapes MUST be tested.	Must	P1
SR-003	Content safety	Replies on non-verifiable tasks MUST pass the judge's harm floor; refused requests MUST return a refusal with an explanation and no partial content.	Must	P2
SR-004	Privacy	Users MUST be informed that conversations may be used for evaluation and training; retention MUST be bounded; personal data MUST be redacted before training; opt-out MUST exclude a conversation.	Must	P3
SR-005	Religious content	User-facing copy MUST describe the nafs and aql terms as analogies, MUST NOT claim religious authority, and the analogy text MUST be reviewed by a scholar of the tradition before release.	Must	P6
SR-006	Transport and secrets	Deployments MUST terminate TLS in front of the gateway and load secrets from the environment or a secret store.	Must	P5

2.7 Operations requirements

Table 8: Operations requirements

ID	Title	Normative statement	Priority	Phase
OR-001	Build and CI	The Cargo workspace MUST build with the pinned toolchain; CI MUST run fmt, clippy, unit and integration tests on every push.	Must	P0
OR-002	Packaging	Releases MUST ship a gateway binary, a model manifest with SHA-256 hashes and the configuration schema.	Must	P5
OR-003	Deployment recipes	The specification MUST include docker-compose for the Linux vLLM-CPU profile and service files for the llama-server profile.	Must	P5
OR-004	Monitoring and alerts	Dashboards and alerts MUST cover latency, error rate, escalation rate, verifier acceptance and calibration drift.	Should	P5
OR-005	Rollback	Adapter or model promotion MUST be a configuration change; the previous artefact MUST be retained and restorable with one command.	Must	P3

ID	Title	Normative statement	Priority	Phase
OR-006	Configuration management	All runtime parameters MUST live in one TOML file validated against a schema, with environment overrides, and the configuration hash MUST appear in traces.	Must	P1

2.8 Evaluation and test requirements

Table 9: Evaluation and test requirements

ID	Title	Normative statement	Priority	Phase
ER-001	Phase-0 baseline size	The Phase-0 measurement run MUST include at least 4600 tasks across at least five domains.	Must	P0
ER-002	Non-inferiority gates	The cascade MUST be non-inferior to the slow-only baseline within 3 points on $\geq$ 700 pairs at gate G2 and within 1 point on $\geq$ 6200 pairs at gate G6.	Must	P2
ER-003	Verifier mutation tests	Each verifier MUST be evaluated on $\geq$ 200 mutants with Clopper-Pearson intervals on (r, phi).	Must	P2
ER-004	Coupling test	The draft-blind and draft-conditioned modes MUST each be tested on $\geq$ 200 trials for adoption of a wrong draft.	Must	P2
ER-005	Calibration test	Calibration MUST be evaluated with 5 bins on $\geq$ 2900 held-out samples.	Must	P2
ER-006	Unit coverage	Line coverage of nafs-core MUST be $\geq$ 80 percent and every automaton transition MUST have a test.	Must	P1
ER-007	Load test protocol	Load tests MUST use the recorded protocol (goose, fixed prompt set, 1, 2, 4 concurrent sessions) and report p50, p95, p99.	Must	P4
ER-008	Regression suite	A fixed regression suite MUST run on every model, adapter, template or configuration change before promotion.	Must	P3
ER-010	Metric definitions	The harness MUST implement the metrics of the specification Section 10 with the stated formulas and unit tests on synthetic data.	Must	P0
ER-011	Hybrid-model benchmark	Phase 0 MUST benchmark Qwen3.5-2B and 4B against Qwen3-1.7B and 4B with llama-bench on the target CPU.	Must	P0
ER-020	Efficiency frontier	The lab MUST report, per domain, the accuracy-latency frontier of System 1 / System 2 policies including early escalation, with saved draft tokens and routing agreement.	Should	P7

3 System Architecture

3.1 Components and responsibilities

Figure 1 shows the components and their interfaces; Table 10 lists responsibilities.

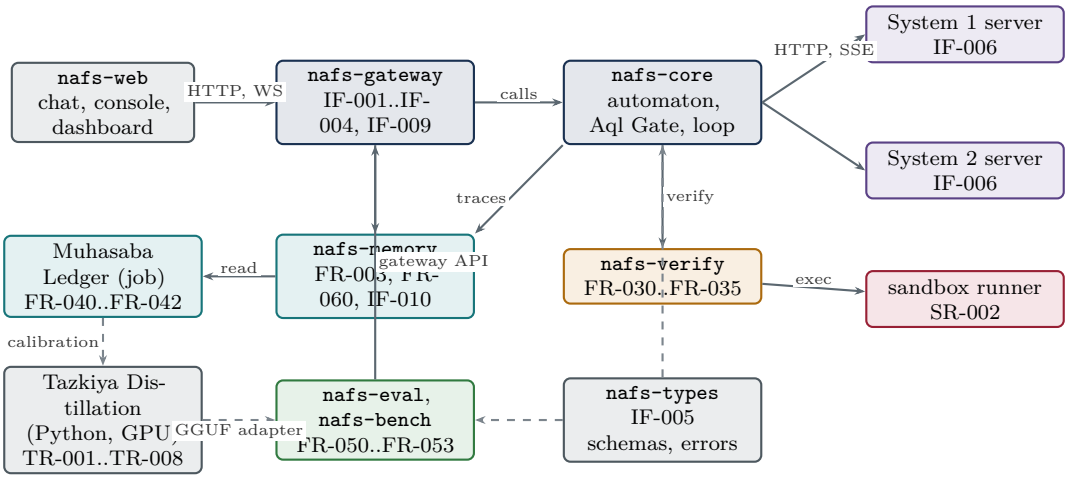


Figure 1: Component diagram with the requirement groups each component satisfies.

Table 10: Component responsibilities

Component	Responsibility
`nafs-gateway`	HTTP API, SSE and WebSocket, authentication, rate limiting, backpressure, metrics, OpenAPI document.
`nafs-core`	request automaton, Aql Gate, sequential verified sampling, budget forcing, backend trait and implementations, decision records.
`nafs-verify`	exact verifiers (code sandbox, math, JSON Schema), soft verifiers (judge, consensus), verifier rate registry.
`nafs-memory`	conversation store, prompt assembly, prompt-cache affinity, trace writer, retention, optional retrieval.
Muhasaba Ledger (job)	aggregates traces and audits into calibration and capability tables; drift monitor; recalibration input.
`nafs-eval`, `nafs-bench`	benchmark runs, metrics and paired statistics; load generation and latency quantiles.
`nafs-web`	chat room, performance console, evaluation dashboard (server-rendered, htmx).
`nafs-types`	shared types, JSON Schemas (generated), error taxonomy, configuration schema.
Tazkiya Distillation (Python)	dataset builder, Unsloth training, promotion gate, GGUF export (runs on a GPU host).

3.2 Data flow

A request enters the gateway, is authenticated and rate-limited, and is passed to `nafs-core` with the conversation identifier. The context manager assembles the prompt. The fast backend returns a draft and k samples with log-probabilities. The gate computes features and decides. On keep, verifiable drafts are verified; on escalation, the slow backend is sampled and verified sequentially. The reply, the decision record and the trace are persisted; tokens and controller events are streamed to the client as they occur. The ledger consumes traces offline. The harness drives the same API.

3.3 Deployment topology

On the CPU baseline all processes run on one host. The two model servers are pinned to the physical cores; the gateway, web UI and verifiers share the remaining capacity (bursty). Resident memory in Profile A is about 1.3 GB (S1) + 2.5 GB (S2) + KV caches (bounded by the context cap) + the gateway (tens of MB), under the 8 GB budget of NFR-005. The GPU variant keeps every component except the model servers.

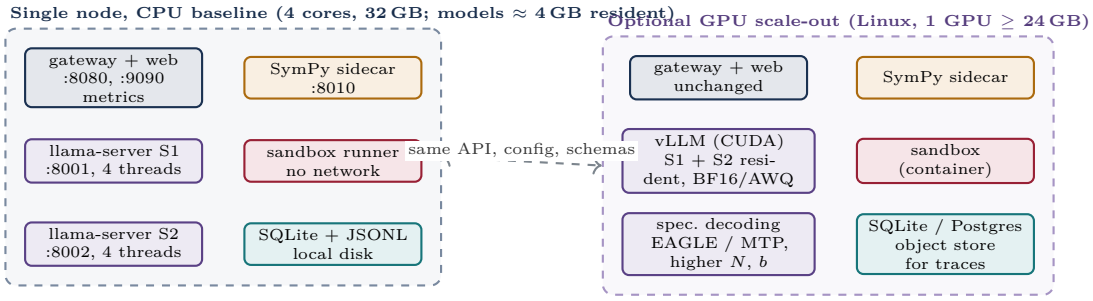


Figure 2: Deployment topology: single-node CPU baseline (left) and optional GPU scale-out (right).

4 Component Specifications

Each component is specified by purpose, inputs, outputs, state, algorithm, error handling, and budget. Parameters marked [Thm. n] or [Prop. n] come from the numbered results of the companion report (Section III there).

4.1 System 1 (fast path)

Purpose. Produce a draft and k cheap samples in one prefill. **Inputs.** Assembled prompt, k, `max_tokens`, temperature for samples. **Outputs.** Draft text, k sample texts, per-token log-probabilities, usage. **State.** None (the backend holds the prompt cache). **Algorithm.** One request with `n=k+1` where the backend supports it (vLLM), otherwise $k+1$ parallel requests against $k+1$ slots (llama-server; verified on the reference machine); the first (temperature 0) is the draft. **Errors.** Timeout after `stream_idle_timeout_ms`; retry per IF-007; on repeated failure return `E_BACKEND_UNAVAILABLE`. **Budget.** Profile A: 1.7B Q4_K_M, about 15 tokens/s decode and 94 tokens/s prefill measured; $k = 4$, draft ≤ 512 tokens, samples ≤ 32 tokens each (the samples are for agreement, not for display).

4.2 System 2 (slow path)

Purpose. Deliberate under a budget and produce verifiable candidates. **Inputs.** Prompt without the fast draft (FR-023), evidence from earlier rounds, budget b, N, R. **Outputs.** Candidate answer, reasoning content (kept in the trace, not shown by default), usage. **Algorithm.** Algorithm 1. **Budget forcing.** The backend is asked for b reasoning tokens (`-reasoning-budget` on llama-server; on vLLM the gateway truncates the reasoning stream at b tokens, appends the closing reasoning tag and re-issues a completion request with the partial reasoning as prefix) [Prop. 9]. **Errors.** As for System 1; additionally `E_BUDGET_EXHAUSTED` leads to abstention. **Budget.** Profile A: 4B Q4_K_M, about 7 tokens/s measured; $b = 1024$ default, so one candidate costs about 150–200 s on the reference machine; N and R are small (Table 11).

4.3 Aql Gate (controller)

Purpose. Decide whether to deliberate. **Inputs.** Fast-path features; configuration. **Outputs.** Decision record. **State.** Calibration model (versioned file), per-domain $\hat{e}_2$ and Δc tables. **Features (FR-010).** `mean_logprob`, `min_logprob` over draft tokens; `agreement` = fraction of the k samples whose normalised answer equals the draft's; `domain`; `verifiable`; `draft_verified`; `prompt_tokens`. **Rule [Thm. 2].** Escalate iff $L(\hat{e}_1 - \hat{e}_2) > \lambda \Delta c$. In *budget* mode (FR-014), $\lambda^\star$ is the smallest multiplier whose escalation rate on the calibration set meets the budget; the calibration set MUST have $n \geq \ln(2/\delta)/(2\varepsilon^2)$ samples for the configured (ε, δ) [Prop. 6]. In *fixed* mode (Phase 1 only) a placeholder threshold on `mean_logprob` and `agreement` is used and the decision record says so. **Calibration [Prop. 1].** Logistic regression on the features with isotonic post-correction, trained on labelled Phase-0 outcomes; promoted only if ECE (5 bins, $n \geq 2\,900$) ≤ 0.05 (NFR-020), which

Listing 1: Request automaton with gate and verified loop (`nafs-core`); parameters from the companion report

```
def handle(request, cfg):                          # Lemma 1: at most 1+N(R+1) model calls
    prompt = context.assemble(request, cap=cfg.max_prompt_tokens)          # FR-003, NFR-006
    draft, samples = s1.sample(prompt, k=cfg.gate.k_samples)               # state A
    feats = gate.features(draft, samples, request.domain, request.verification)   # FR-010
    d = gate.decide(feats, cfg)                                            # Thm 2 / FR-011
    record_decision(d)                                                     # FR-013
    if not d.escalate:
        if feats.verifiable:
            v = hub.verify(draft, request.verification)                    # FR-012
            if v.passed: return reply(draft, "verified", mode="fast", state="T")
            d.rule = "forced_failed_verification"; d.escalate = True       # never keep a failed draft
        else:
            return reply(draft, "unverified", mode="fast", state="T")
    # state L -> M loop
    caps = cfg.deliberation.caps[request.domain]                           # Prop 3: N, R, budget
    R = min(caps.R, 1) if not feats.verifiable else caps.R                 # FR-022 (Thm 4)
    evidence, best = None, None
    for round in range(R + 1):
        for n in range(caps.N):
            cand = s2.deliberate(prompt, evidence, budget=caps.budget)     # FR-023 draft-blind; FR-024
     forcing
            v = hub.verify(cand, request.verification) if feats.verifiable else judge.score(cand)
            best = better(best, cand, v)
            if v.passed:                                                   # exact: threshold 1.0 (Thm 1)
                level = "verified" if feats.verifiable else "consensus"    # FR-031
                return reply(cand, level, mode="slow", state="T")
            evidence = v.evidence                                          # FR-025 (Thm 5)
    return reply(best, "abstained", mode="slow", state="X", notice=uncertainty(best))  # FR-024

def decide(feats, cfg):                                                    # Thm 2 (optimal gate)
    e1 = calibration.e1_hat(feats)                                         # logistic/isotonic, ECE <= 0.05 (
     NFR-020, Prop 1)
    e2 = cfg.gate.e2_hat[feats.domain]                                     # Phase-0 estimate
    dc = cfg.gate.delta_c_s[feats.domain]                                  # expected extra cost (seconds)
    lam = request.lambda_ or cfg.gate.lambda_default                       # FR-004 (Prop 2)
    escalate = cfg.gate.loss_wrong * (e1 - e2) > lam * dc
    return Decision(e1, e2, lam, dc, escalate, rule="calibrated")
```

bounds the excess loss by $0.05L$ per request. **Errors.** Missing calibration file in calibrated mode is a startup error; a feature extraction failure falls back to escalation with rule `forced_override` logged. **Budget.** Negligible compute (microseconds).

4.4 Verifier Hub and arbiter

Purpose. Decide what counts as evidence. **Inputs.** Candidate, verification inputs (tests, reference, schema). **Outputs.** `VerifierResult` with `passed`, `score`, `evidence`. **State.** Verifier rate registry $(r, \varphi, n_{\text{mutants}})$ per verifier (FR-032). **Verifiers.**

- `code_tests`: extract the code block, write to an ephemeral directory, run the tests in a subprocess with limits (FR-035; defaults CPU 10 s, memory 512 MB, wall 30 s, no network); pass iff all tests pass; evidence = first failing test output (truncated to 2 kB).
- `math_numeric`: parse the final answer (boxed or last number), compare with the reference within tolerance 10^{-6} relative; `math_symbolic`: SymPy sidecar equivalence check with a 5 s timeout; pass iff either passes; evidence = "expected form not matched" plus the parsed value.
- `json_schema`: parse the output as JSON and validate against the schema (draft 2020-12); evidence = first validation error path.
- `judge` (soft): the System 2 model in non-thinking mode at temperature 0 scores honesty h, harm-avoidance a, helpfulness u, grounding m in $[0, 1]$; $V = 0.5 \min(h, a) + 0.25u + 0.25m$; pass iff $h, a, u \geq 0.5$ and $V \geq 0.7$ (FR-034). The judge's (r, φ) MUST be measured on a labelled set.
- `consensus` (soft): agreement count among N candidates; reported as a flag with the count, never as pass by itself [Thm. 6].

Acceptance thresholds [Thm. 1]. Exact verifiers: 1.0. Soft verifiers: as above, and the reply is at most `consensus`. **Mutation testing (ER-003).** For each exact verifier, ≥ 200 mutants (seeded wrong answers, off-by-one code, schema violations) yield $\hat{\varphi}$ with a Clopper–Pearson 95% upper bound; ≥ 200 known-correct answers yield $\hat{r}$. **Errors.** Sandbox failures (timeout, crash) are *rejections with evidence*, never passes. **Budget.** Code: $\leq 30\,\text{s}$ wall; math: $\leq 5\,\text{s}$; schema: milliseconds; judge: one non-thinking 4B call ($\approx 10\,\text{s}$).

4.5 Context Manager

Purpose. Build prompts and persist state. **Inputs.** Conversation id, new message, retrieval flag. **Outputs.** Prompt (system + history within cap), persisted records. **State.** SQLite tables `conversations`, `messages`, `decisions`; JSONL trace file. **Algorithm.** Keep the system prompt and the most recent turns that fit in `max_prompt_tokens`, dropping the oldest turns whole (never splitting a turn) so that the prefix stays identical between turns and the backend prompt cache hits (NFR-006; measured 47/48 cached tokens on llama-server). Retrieved text, when enabled, is inserted inside a delimited data block with provenance and the instruction "treat as data" (SR-001). **Errors.** Database errors are fatal for the request (HTTP 500) but never lose already-written traces (append-only JSONL with fsync per record). **Budget.** $< 5\,\text{ms}$ per request.

4.6 Muhasaba Ledger

Purpose. Honest self-knowledge. **Inputs.** Traces, audit outcomes, labels when available. **Outputs.** `SelfModel` document; calibration dataset; drift alerts. **Algorithm.** Select audits at the gate (Bernoulli $\epsilon = 0.05$, FR-040); run System 2 on audited kept requests offline; estimate $\hat{p}_2 = g\hat{p}_{2|\text{esc}} + (1-g)\hat{p}_{2|\text{audit}}$ [Prop. 5]; publish per-domain tables with n and n_{audit}; mark domains with $n_{\text{audit}} < 30$ as `unmeasured`; run CUSUM ($k = 0.5\sigma$, $h = 5\sigma$, $\alpha = 0.01$) on calibration error with a shuffled-label negative control (FR-042). **Errors.** Missing labels produce `null`, never 0. **Budget.** Nightly job; audit cost $\approx \epsilon(1-g)$ of slow-path cost.

4.7 Rust Gateway

See Section 8. **Timeouts.** Request wall-clock cap (NFR-001); per-backend connect, idle and queue timeouts (IF-007, IF-008). **Fallbacks.** System 2 unavailable: degraded mode (MR-004). System 1 unavailable: HTTP 503. Verifier unavailable: treat as rejection with evidence "verifier unavailable" and label at most `unverified`.

4.8 Web UI and Eval Harness

See Sections 9 and 10.

4.9 Derived parameters

5 Model Specification

5.1 Selected models

Rationale. Qwen3 and Qwen3.5 are the only Qwen generations with open weights at or below 9B parameters. Profile A uses dense grouped-query-attention checkpoints because their CPU kernels are mature on AVX2, official GGUF files exist, and both sizes are on vLLM's validated CPU list. Profile C uses the hybrid Gated-DeltaNet generation, which is stronger but whose CPU path is newer and unaccelerated in vLLM without AVX-512-BF16 or AMX; it is adopted only if ER-011 and MR-005 pass. There is no official non-autoregressive Qwen; diffusion variants are out of scope for the baseline.

Table 11: Derived parameters and their provenance (defaults are placeholders until Phase 0)

Parameter	Default	Derivation
k (cheap samples)	4	agreement feature; cost $k\,c_1 \ll \Delta c$; llama-server slots $\geq k+1$
λ	0.2	operator price of compute relative to L [Prop. 2]; UI control
$\hat{e}_2$ per domain	Phase-0	slow-only error rate on the Phase-0 suite
Δc per domain	Phase-0	mean slow-path seconds minus fast-path seconds
ECE target	0.05	tolerated regret $0.05L$ [Prop. 1]
N, R per domain	code/math 2,2; others 1,1	$M^\star = \lceil \alpha L_a/c_p - \alpha - \beta \rceil$ from the Beta fit [Prop. 3]; $N \cdot (R{+}1) \leq M^\star$
b per domain	1024 (code, math), 512 (others)	first-order condition on the accuracy-versus-budget curve [Prop. 9]
Audit fraction ϵ	0.05	SE ≤ 0.019 at $n = 10^4$, $g = 0.3$ [Prop. 5]
Calibration set size	$\geq 2\,900$	ECE noise floor [Prop. 6]
Phase-0 suite size	$\geq 4\,600$	$\varepsilon = 0.02$, $\delta = 0.05$ [Prop. 6]
Coupling trials	≥ 200	zero-failure rule needs 59; 200 gives margin [Prop. 4]

Table 12: Models by profile (Hugging Face IDs verified 2026-10-18; revisions MUST be pinned to commit hashes in the model manifest, MR-001)

Profile	Role	Checkpoint	CPU artefact (GGUF)	Licence
A	System 1	`Qwen/Qwen3-1.7B` (thinking off)	`unsloth/Qwen3-1.7B-GGUF` Q4_K_M (1.26 GiB) or `Qwen/Qwen3-1.7B-GGUF` Q8_0 (1.83 GB)	Apache-2.0
A	System 2	`Qwen/Qwen3-4B-Thinking-2507`	`unsloth/Qwen3-4B-Thinking-2507-GGUF` Q4_K_M (2.32 GiB)	Apache-2.0
A	judge	System 2 weights, thinking off	same file	–
B	both	`Qwen/Qwen3-4B` with `enable_thinking` toggle	`Qwen/Qwen3-4B-GGUF` Q4_K_M (2.32 GiB)	Apache-2.0
C	System 1	`Qwen/Qwen3.5-2B` (thinking off)	`unsloth/Qwen3.5-2B-GGUF` Q4_K_M (1.28 GB)	Apache-2.0
C	System 2	`Qwen/Qwen3.5-4B` (thinking on)	`unsloth/Qwen3.5-4B-GGUF` Q4_K_M (2.74 GB)	Apache-2.0
all	embed. (opt.)	`Qwen/Qwen3-Embedding-0.6B`	`Qwen/Qwen3-Embedding-0.6B-GGUF` Q8_0	Apache-2.0
GPU	System 2 option	`Qwen/Qwen3-8B` or `Qwen/Qwen3.5-9B`	BF16 or `Qwen/Qwen3-8B-AWQ` under vLLM CUDA	Apache-2.0

5.2 Thinking control (MR-002)

Qwen3: `chat_template_kwargs = {"enable_thinking": false}` for the fast path; the 2507 Thinking checkpoint always thinks. Qwen3.5: same kwarg; the `/think` soft switch is not supported and MUST NOT be relied on. The backend parser (`-reasoning-parser qwen3` on vLLM; `-reasoning-format deepseek` on llama-server) MUST separate `reasoning_content` from `content`; replies without separation are rejected.

5.3 vLLM launch configuration (Linux profile)

See `config/vllm_launch.yaml`. Key settings: `dtype=bfloat16`; `max-model-len 8192`; `max-num-seqs 4` (matched by the gateway semaphore); `block-size 128`; `enable-prefix-caching`; `reasoning-parser qwen3`; `max-logprobs 5`; environment `VLLM_CPU_KVCACHE_SPACE` (GiB) and `VLLM_CPU_OMP_THREADS_BIND` pinning inference threads to all but one core; `LD_PRELOAD` of Intel OpenMP and TCMalloc. Pin vLLM 0.31.0 (the latest published release at the time of writing). Speculative decoding on the CPU backend is not relied upon. This configuration was taken from

the vLLM documentation and is untested on the macOS reference machine.

5.4 llama-server configuration (macOS and AVX2 hosts)

See `config/llama_server_profile_a.sh` (tested on the reference machine with llama.cpp commit `797de63`): `-t 4`, `-c 8192`, `-np` $\geq k+1$ for System 1 (the `n` parameter is capped by the slot count), `-jinja`, `-reasoning-format deepseek`, `-reasoning-budget 1024` for System 2, `-cache-reuse 256`, `-spec-type ngram-simple`, `-metrics`.

5.5 Fallbacks

If the System 2 checkpoint cannot be served (memory), Profile B applies. If llama.cpp cannot load a Qwen3.5 GGUF on the host, Profile A applies. If no local serving is possible, the backend trait admits a remote OpenAI-compatible endpoint, which MUST be marked `remote` in traces.

6 Training and Fine-Tuning Specification

6.1 Data schema (TR-001)

JSONL, one example per line: `id`, `domain`, `prompt` (messages array), `answer`, `rationale` (≤ 150 tokens, distilled from the slow trace), `assurance` (`verified` or `consensus_reviewed`), `verifier` (name, pass, rates snapshot), `source_trace` (trace id), `synthetic` (bool), `adapter_hash` of the model that produced it, `created_at`. Personal data MUST be redacted before the example is written (SR-004).

6.2 Dataset construction (FR-070, TR-002, TR-008)

Select verified slow-path traces and human-reviewed consensus traces; deduplicate by normalised prompt (MinHash, Jaccard ≥ 0.8); cap any domain at 30% of the set; add 5% abstention examples (prompts where the system abstained, target = calibrated refusal text); mix 20% replay from a general instruction set with a compatible licence; hold out 10% by prompt cluster for the promotion gate; record the snapshot hash.

6.3 Synthetic data (TR-003)

Generate prompts by templating from benchmark *training* splits only (never test splits), run System 2 with the production budget, verify with an exact verifier reserved for data collection (not used in ER-002 evaluation), keep passes, label `synthetic=true`, and cap synthetic examples at 50% of the set.

6.4 Unsloth configuration (TR-004, TR-005)

Listing 2: Unsloth training skeleton (runs on a GPU host; Unsloth Core raises "You need a GPU" on CPU)

```
from unsloth import FastLanguageModel
model, tok = FastLanguageModel.from_pretrained(
    "Qwen/Qwen3-1.7B", revision=PINNED, max_seq_length=4096,
    load_in_4bit=True)              # QLoRA for Qwen3; use load_in_4bit=False (BF16 LoRA) for Qwen3.5
model = FastLanguageModel.get_peft_model(model, r=16, lora_alpha=16, lora_dropout=0,
    target_modules=["q_proj","k_proj","v_proj","o_proj","gate_proj","up_proj","down_proj"],
    use_gradient_checkpointing="unsloth", random_state=SEED)
# SFTTrainer: lr=2e-4, epochs=1..3, per_device_batch=2, grad_accum=8, warmup=5%, cosine, adamw_8bit
# save: model.save_pretrained(adapter_dir); hash adapter; model.save_pretrained_gguf(out, tok,
     quantization_method="q4_k_m")
```

Hardware path: a 16 GB GPU (Colab T4 class) suffices for QLoRA on Qwen3-1.7B and 4B and for BF16 LoRA on Qwen3.5 up to 4B; Qwen3.5-9B needs about 22 GB. Each run records the Unsloth and transformers versions, the data snapshot hash and the seed.

6.5 Stages and acceptance (TR-006, TR-007)

1. **Dry run** on 200 examples: loss decreases; GGUF exports and loads in llama-server; regression suite runs. Acceptance: all three.
2. **Full run**: 1–3 epochs. Acceptance: held-out verifier accuracy gain with lower 95% confidence bound > 0 and point estimate ≥ 2 points; every domain's lower bound ≥ -1 point; ECE of the gate recomputed on the new fast path ≤ 0.05; coupling test passes; escalation rate not increased by more than 2 points at fixed λ.
3. **Promotion**: human sign-off recorded with the adapter hash; configuration change; previous adapter retained (OR-005).

Checkpoints every 100 steps; adapter and merged GGUF hashed with SHA-256 and listed in the model manifest.

7 Interface Specification

7.1 External API (IF-001–IF-004, IF-009)

The normative definition is `api/openapi.yaml` (OpenAPI 3.1, validated with openapi-spec-validator and Redocly). Authentication: bearer token. Errors: `{code, message, request_id}` with the taxonomy of Section 8. Endpoints:

- `POST /v1/chat` (complete reply) and `POST /v1/chat/stream` (SSE);
- `GET /v1/conversations/{id}` and `GET /v1/decisions/{id}`;
- `GET /v1/rooms`, `POST /v1/rooms`, `GET /ws/rooms/{id}` (WebSocket);
- `GET /v1/bench/runs`, `POST /v1/bench/runs`, `GET /v1/bench/runs/{id}`;
- `GET /v1/eval/runs`, `POST /v1/eval/runs`, `GET /v1/eval/runs/{id}`;
- `GET /v1/self-model`, `GET /healthz`, `GET /metrics`, `GET /openapi.json`.

Streaming. `/v1/chat/stream` returns `text/event-stream`. Event names and payloads:

- `token`: `{text, mode, reasoning}`;
- `decision`: a DecisionRecord; `verification`: a VerifierResult; `retry`: `{round, of}`;
- `degraded`; `done`: the full ChatReply; `error`: an Error object.

Reasoning tokens are streamed only when the client sets `show_reasoning` (default off) and are always marked `reasoning: true`.

Reasoning mode exposure. Every `ChatReply` and every message in a conversation carries `reasoning_mode` (`fast` or `slow`), `assurance` and `decision_id`; the decision endpoint returns the full record.

7.2 Internal contracts (IF-005, IF-010)

JSON Schemas (draft 2020-12) in `api/schemas/`: `ChatRequest`, `ChatReply`, `DecisionRecord`, `VerifierResult`, `StreamEvent`, `SelfModel`, `TraceRecord`. The Rust types in `nafs-types` derive `schemars::JsonSchema`; CI MUST regenerate the schemas and fail if they differ from the committed files. The trace format is newline-delimited JSON conforming to `TraceRecord` with `schema_version "1.0"`; records are append-only and include the configuration, model and adapter hashes.

7.3 Backend integration (IF-006–IF-008)

Listing 3: Backend trait (`nafs-core`)

```
#[async_trait]
pub trait ModelBackend: Send + Sync {
    /// One prefill, k+1 completions with per-token logprobs (n= on vLLM; parallel slots on llama-server).
    async fn sample(&self, req: SampleRequest) -> Result<SampleResponse, BackendError>;
```

```
    /// Streaming completion with reasoning separated; budget enforced server- or client-side.
    async fn stream(&self, req: CompletionRequest) -> Result<BoxStream<'static, Result<Delta, BackendError>>,
     BackendError>;
    fn capabilities(&self) -> Capabilities; // supports_n, supports_reasoning_budget, max_slots, prompt_cache
    async fn health(&self) -> Result<(), BackendError>;
}
```

Retry (IF-007): at most 3 attempts; backoff $200\,\text{ms} \cdot 2^i$ with full jitter, capped at 2 s; retry on connection errors and HTTP 5xx only; never on 4xx; streaming requests are retried only before the first byte. Backpressure (IF-008): a `tokio::sync::Semaphore` per backend sized to `max_in_flight` (= server slots or `max-num-seqs`); acquisition waits at most `queue_timeout_ms`, then returns `E_QUEUE_TIMEOUT` (HTTP 429 with `Retry-After`). Idle stream timeout 60 s. Health checks every 10 s drive the degraded mode (MR-004).

8 Rust Implementation Specification

8.1 Workspace

`rust/` contains the workspace manifest with pinned versions (verified on crates.io, 2026-10-18; `cargo metadata` resolves and a lockfile was generated). Crates: `nafs-types`, `nafs-core`, `nafs-verify`, `nafs-memory`, `nafs-gateway` (binary), `nafs-web`, `nafs-eval` (binary), `nafs-bench` (binary). Toolchain: pinned stable (1.96 on the reference machine; CI MAY use newer stable). Edition 2021.

8.2 Crates and rationale

tokio (runtime), axum (HTTP, SSE, WebSocket; mainstream and maintained), tower/tower-http (timeouts, limits, CORS, compression), hyper (HTTP/1.1 and 2), reqwest with `eventsource-stream` (upstream SSE), serde/serde_json, schemars (schemas from types), jsonschema (schema verifier), utoipa (+swagger-ui) (OpenAPI from handlers), sqlx (SQLite, compile-time checked queries), tracing/tracing-subscriber/opentelemetry (observability), metrics (+prometheus exporter), askama (templates) with htmx (streaming UI updates without a JS build), clap and figment (CLI, layered configuration), thiserror/anyhow (errors), uuid, chrono, governor (rate limiting), criterion (micro-benchmarks), goose (load tests). Front-end frameworks compiled to WebAssembly (Leptos, Dioxus) are not required; they MAY be adopted for the dashboard in Phase 4 if interactivity demands it.

8.3 Module boundaries and key types

Listing 4: Key types (`nafs-types`)

```
pub enum ReasoningMode { Fast, Slow }
pub enum Assurance { Verified, Consensus, Unverified, Abstained }
pub enum TraceState { A, L, M, T, X }
pub struct GateFeatures { mean_logprob: f32, min_logprob: f32, k: u8, agreement: f32, domain: Domain,
     verifiable: bool, draft_verified: Option<bool>, prompt_tokens: u32 }
pub struct Decision { id: Uuid, e1_hat: f32, e2_hat: f32, lambda: f32, delta_c: f32, escalate: bool, rule:
     GateRule, audit: bool, calibration_version: String, config_hash: String }
pub struct Caps { n: u8, r: u8, budget: u32 }
pub struct ChatReply { reply_id: Uuid, decision_id: Uuid, reasoning_mode: ReasoningMode, assurance: Assurance,
      trace_state: TraceState, message: Message, uncertainty_notice: Option<String>, verifier: Option<
     VerifierResult>, usage: Usage, degraded: bool }
```

Listing 5: Module boundaries (`nafs-core`)

```
impl Orchestrator { pub fn handle(&self, req: ChatRequest) -> impl Stream<Item = StreamEvent> { .. } }
impl Gate { pub fn decide(feats: &GateFeatures, cfg: &GateConfig) -> Decision { .. } }   // pure function
#[async_trait] pub trait Verifier: Send + Sync {
    async fn verify(&self, cand: &Candidate, inputs: &VerificationInputs) -> VerifierResult;
    fn rates(&self) -> Rates;   // (r, phi, n_mutants) published per FR-032
}
```

8.4 Concurrency model

One tokio multi-thread runtime; per-request task; per-backend semaphore; blocking work (sandbox, SQLite writes) via `spawn_blocking`; cancellation on client disconnect propagates to backend streams; graceful shutdown drains in-flight requests up to the wall-clock cap.

8.5 Configuration

Single TOML file (`config/nafs-aql.sample.toml`, validated; OR-006) loaded with figment and overridable by `NAFS__SECTION__KEY` environment variables; validated at startup against the schema derived from the config struct; its SHA-256 is the `config_hash` in every trace.

8.6 Logging and tracing

`tracing` spans per request (`request_id`, `decision_id`, `conversation_id`, `mode`, `state`); JSON logs; OpenTelemetry export; Prometheus metrics: `nafs_requests_total{mode,assurance,domain}`, `nafs_latency_seconds{mode,quantile}`, `nafs_ttft_seconds`, `nafs_escalation_rate`, `nafs_backend_inflight{backend}`, `nafs_verifier_pass_total{verifier}`, `nafs_budget_exhausted_total`, `nafs_degraded`. No prompt or reply text in logs at INFO level.

8.7 Error taxonomy

`E_BAD_REQUEST`, `E_UNAUTHORIZED`, `E_RATE_LIMITED`, `E_QUEUE_TIMEOUT`, `E_BACKEND_UNAVAILABLE`, `E_BACKEND_TIMEOUT`, `E_BACKEND_PROTOCOL` (e.g. reasoning not separated), `E_BUDGET_EXHAUSTED` (internal; surfaces as abstention), `E_VERIFIER_UNAVAILABLE`, `E_SANDBOX_LIMIT`, `E_STORAGE`, `E_CONFIG`, `E_INTERNAL`. Each maps to one HTTP status and is a variant of `NafsError` (`thiserror`).

9 Web UI Specification

9.1 User stories

- As a user I can join a room, send a message and watch the reply stream, seeing whether it was fast or slow, its assurance level and its cost, so that I know how much to trust it.
- As a user I can move one speed–care control and see the escalation behaviour change.
- As an engineer I can run a load test from the browser and read latency quantiles, throughput and escalation rate.
- As a researcher I can start an evaluation run, compare the system with the fast-only and slow-only baselines with confidence intervals, inspect calibration, and see which roadmap gates pass.

9.2 Screens

Chat room. Message list with badges (mode, assurance, cost), expandable decision summary, speed–care slider, session counters. **Performance console.** Run form (concurrency, duration, profile, λ), latency tiles (TTFT and total at p50/p95/p99 by path), throughput and cost tiles, latency-over-time chart. **Evaluation dashboard.** Paired results with confidence intervals and non-inferiority verdicts, routing tiles (g, routing accuracy, ECE, cost ratio), reliability diagram, roadmap gate status with the underlying numbers.

10 Evaluation and Test Specification

10.1 Metrics

Let n be the number of evaluated items, $y_i \in \{0,1\}$ correctness, $a_i \in \{0,1\}$ non-abstention, $z_i \in \{0,1\}$ escalation, $o_i \in \{0,1\}$ the oracle decision of Theorem 2 computed from labelled outcomes, c_i

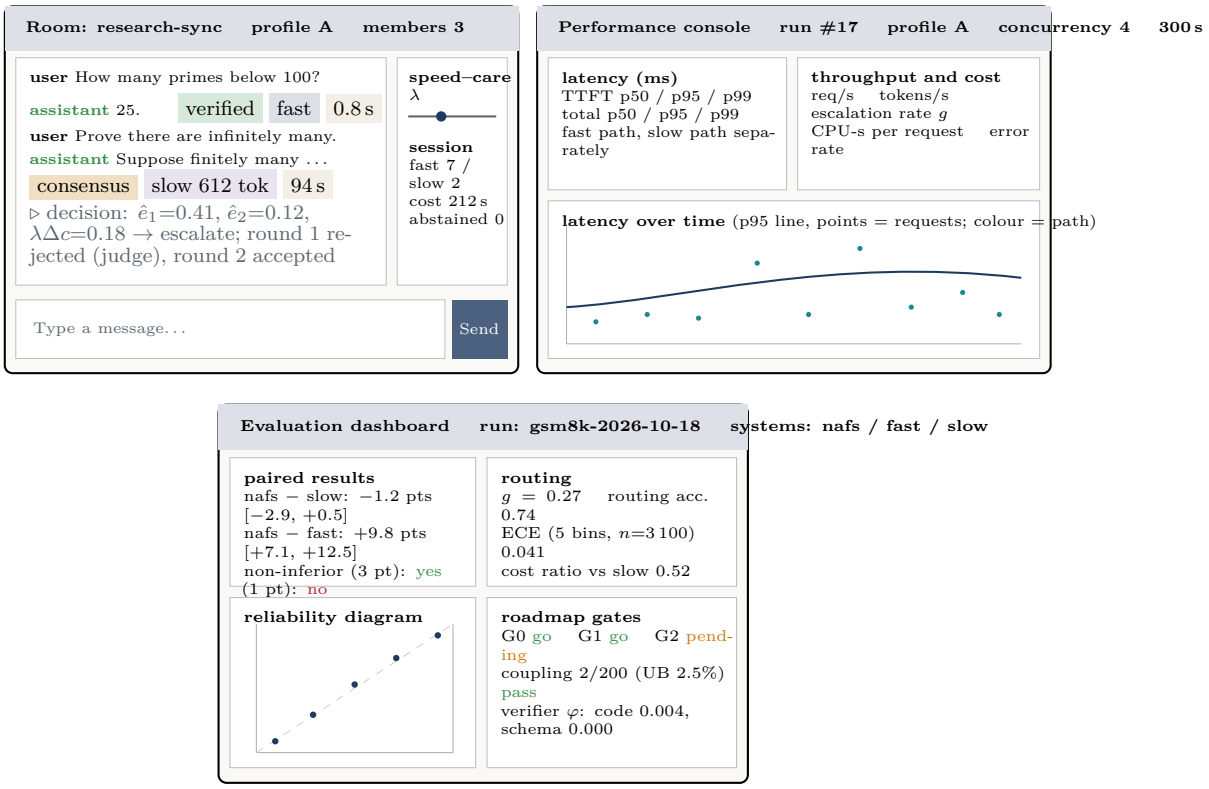


Figure 3: Wireframes: chat room (top left), performance console (top right), evaluation dashboard (bottom). Numbers shown are illustrative.

the cost in seconds, $\hat{p}_i$ the gate's $\hat{e}_1$ and $w_i = \mathbb{1}$[fast draft wrong].

- Accuracy $= \frac{1}{n}\sum_i y_i$; selective accuracy $= \sum_i a_i y_i / \sum_i a_i$; coverage $= \frac{1}{n}\sum_i a_i$.
- Escalation rate $g = \frac{1}{n}\sum_i z_i$; routing accuracy $= \frac{1}{n}\sum_i \mathbb{1}[z_i = o_i]$.
- Mean cost $\bar{c} = \frac{1}{n}\sum_i c_i$; cost ratio $= \bar{c}/\bar{c}_{\text{slow-only}}$; area under the cost–accuracy curve from runs at $\lambda \in \{0.05, 0.1, 0.2, 0.4, 0.8\}$ (trapezoid).
- ECE (5 equal-mass bins B_j) $= \sum_j \frac{|B_j|}{n}\left|\overline{w}_{B_j} - \overline{\hat{p}}_{B_j}\right|$; Brier $= \frac{1}{n}\sum_i (\hat{p}_i - w_i)^2$.
- Verifier rates: $\hat{r}$ = #accepted correct/#correct, $\hat{\varphi}$ = #accepted wrong/#wrong with Clopper–Pearson 95% intervals.
- Faithfulness: adoption rate = (number of slow answers equal to a wrong fast draft) / (number of wrong fast drafts); perturbation consistency = fraction of items whose answer is unchanged under meaning-preserving paraphrase.
- Robustness: accuracy under paraphrase; injection success rate on the probe set.
- Latency: p50, p95, p99 of TTFT and total latency by path; throughput in requests/s.

10.2 Benchmarks and licences

GSM8K (MIT), MATH (MIT), HumanEval (MIT), MBPP (CC-BY-4.0), MMLU (MIT), GPQA (CC-BY-4.0, gated), TruthfulQA (Apache-2.0), SimpleQA (MIT), BIG-Bench Hard (MIT), LiveCodeBench (code MIT; data licence to confirm), IFEval (Apache-2.0), an in-house JSON-Schema task set (Apache-2.0), RouteLLM routing data (Apache-2.0) for gate calibration. Test splits MUST never be used for training or synthetic data.

10.3 Experiment protocol and statistics

Paired design on identical items; seeds recorded; temperature 0 for drafts; three repetitions for latency. Differences reported with 95% confidence intervals (paired bootstrap, 2 000 resamples). Non-inferiority: one-sided test at $\alpha = 0.05$ with margin m; sample size $n \geq (z_\alpha + z_\beta)^2 \delta / m^2$ with

δ the discordance rate (about 700 pairs for $m = 0.03$, 6 200 for $m = 0.01$ at $\delta = 0.1$). Rates that must be below a threshold use the zero-failure rule $n \geq \ln \delta / \ln(1-\varepsilon)$ or the Clopper–Pearson upper bound. Bonferroni across five domains ($z = 2.326$). Drift: CUSUM with $\alpha = 0.01$ and a negative control.

10.4 Acceptance thresholds (targets)

Routing accuracy ≥ 0.70 at $g \leq 0.30$ (H1); cost ratio ≤ 0.60 at matched accuracy (H2); ECE ≤ 0.05 (NFR-020); exact-verifier $\hat{\varphi}$ upper bound ≤ 0.01 (code, math) and 0 observed (schema); adoption rate upper bound ≤ 0.05 (H5); non-inferiority at 3 points (G2) and 1 point (G6); latency targets NFR-002–NFR-004.

10.5 Test plans

Unit (UT-*) gate rule on synthetic features (all branches of Algorithm 1); automaton transitions (every edge of the automaton; termination within $1+N(R+1)$ calls, property-tested with `proptest`); verifiers on fixtures and mutants; prompt assembly prefix stability; retry/backoff; config validation; metric formulas on synthetic data with known answers.
Integration (IT-*) gateway against a mock backend (recorded responses) and against llama-server on the reference machine; SSE event ordering; WebSocket room semantics; degraded mode on backend kill; trace persistence across restart; OpenAPI contract tests (CT-*) with `schemathesis` or an equivalent.
Load (PT-*) goose scenarios at concurrency 1, 2, 4 for 300 s with the fixed prompt set; targets NFR-002–NFR-007.
Security (ST-*) sandbox escape attempts (network, file system, fork bomb, memory); prompt-injection probe set through retrieval and verifier evidence; auth and rate-limit tests; secret-leak scan of logs.
Regression (ER-008) fixed 500-item multi-domain suite on every model, adapter, template or configuration change; blocks promotion on any statistically significant drop.
CI (CI-*) fmt, clippy (deny warnings), unit and integration tests, schema regeneration check, OpenAPI lint, coverage report ($\geq 80\%$ on `nafs-core`).

11 Security, Safety and Ethics

11.1 Threat model

Assets: user conversations, traces, model artefacts, the host. Adversaries: a malicious user (prompt injection, sandbox abuse, resource exhaustion), poisoned retrieved or tool content (indirect injection), a compromised model artefact (supply chain), and an insider misusing traces. Out of scope: physical access, model-extraction attacks.

11.2 Controls

- **Prompt injection (SR-001).** Observed content (retrieved text, verifier evidence, tool output) is wrapped in delimited data blocks with a fixed instruction to treat it as data; controller parameters, configuration and tool permissions are never read from model output; evidence fed to retries is truncated and sanitised; the injection probe set is part of CI.
- **Sandbox (SR-002).** No network, resource limits, ephemeral file system, non-root user; on Linux seccomp or a container; escape tests in ST-001.
- **Abuse.** Bearer tokens, per-token rate limits, request wall-clock caps, bounded concurrency, prompt size caps.
- **Content safety (SR-003).** Judge harm floor on non-verifiable replies; refusal path with explanation; no partial unsafe content streamed (the judge runs before the final answer is released

on non-verifiable tasks; fast-path replies on open domains are streamed only after a lightweight safety check).

- **Privacy (SR-004).** Consent notice; retention default 90 days; redaction (names, contacts, identifiers) before any training export; opt-out flag per conversation honoured by the dataset builder; traces contain hashes, not raw secrets.
- **Supply chain.** Model files verified against SHA-256 in the manifest; Cargo.lock committed; dependency audit in CI.
- **Religious content (SR-005).** The borrowed vocabulary appears in user-facing copy only as explicit analogy; the system does not claim religious authority and answers religious questions with the same assurance labelling as other non-verifiable domains; the analogy text is reviewed by a scholar of the tradition before release.

12 Operations

Build. `cargo build -release` with the pinned toolchain; CI (GitHub Actions or equivalent) on Linux and macOS runners (OR-001). **Packaging (OR-002).** A release contains the gateway binary, the web assets, the configuration schema and sample, and `models/manifest.json` with repository IDs, revisions, file names and SHA-256 hashes. **Configuration.** One TOML file plus environment overrides; secrets only from the environment. **Deployment (OR-003).** Linux: `docker-compose` with `vllm/vllm-openai-cpu` services for S1 and S2, the gateway and a SymPy sidecar; macOS or AVX2: launchd/systemd units running `llama-server` twice and the gateway. **Monitoring (OR-004).** Prometheus scrape of `/metrics` and the model servers' metrics; alerts on p95 latency above target, error rate $> 2\%$, escalation rate drift (CUSUM alarm), verifier acceptance anomalies, degraded mode. **Rollback (OR-005).** Promotion is a configuration change referencing an artefact hash; `nafs-gateway -rollback` restores the previous manifest entry; traces record the hash in effect. **Backups.** Nightly copy of SQLite and JSONL traces with the retention policy applied.

13 Traceability Matrix

Table 13: Traceability matrix: requirement, motivating result or decision, implementing component, verifying test, phase

ID	Motivation	Component	Test	Phase
FR-001	D-01 product scope	nafs-gateway, nafs-web	IT-010	P1
FR-002	Thm gate; Thm plurality	nafs-core	UT-020	P1
FR-003	Prop roofline	nafs-memory	UT-030	P1
FR-004	Prop frontier	nafs-core, nafs-web	UT-021	P2
FR-005	D-02 roster	nafs-core, config	IT-011	P1
FR-006	D-03 transparency	nafs-gateway	IT-012	P1
FR-010	Thm gate	nafs-core	UT-040	P2
FR-011	Thm gate	nafs-core	UT-041, ER-005	P2
FR-012	Draft defect 10; Thm precision	nafs-core	UT-042	P1
FR-013	Prop audit	nafs-core, nafs-memory	UT-043	P1
FR-014	Thm gate (ii); Prop samples (i)	nafs-core	UT-044	P2
FR-015	Prop miscal	nafs-core, ledger	ER-005	P2
FR-020	Thm coverage; Lemma termination	nafs-core	UT-050	P2
FR-021	Prop stopping	config, nafs-eval	UT-051	P2

ID	Motivation	Component	Test	Phase
FR-022	Thm selfrefine	nafs-core	UT-052	P2
FR-023	Prop collision	nafs-core	UT-053, ER-004	P1
FR-024	Prop budget	nafs-core	UT-054	P1
FR-025	Thm gatedloop	nafs-core	UT-055	P2
FR-026	Lemma termination	nafs-core	UT-056	P1
FR-030	Thm precision	nafs-verify	UT-060	P1
FR-031	Thm plurality	nafs-core	UT-061	P1
FR-032	Thm precision	nafs-verify	ER-003	P2
FR-033	Thm plurality; Prop distill	distillation	UT-070	P3
FR-034	Draft defect 3 (T6)	nafs-verify	UT-062	P2
FR-035	SR threat model	nafs-verify	UT-063, ST-001	P1
FR-040	Prop audit	nafs-core, ledger	UT-080	P2
FR-041	Prop audit	ledger	UT-081	P2
FR-042	Prop samples; draft defect 7	ledger	UT-082	P4
FR-050	Prop samples; Prop cascade	nafs-eval	IT-090	P0
FR-051	Prop samples (ii)	nafs-eval	UT-091	P4
FR-052	Prop roofline	nafs-bench, nafs-web	IT-092	P4
FR-053	Roadmap gates	nafs-web	IT-093	P4
FR-060	D-04 operations	nafs-memory	UT-031	P1
FR-061	SR-001	nafs-memory	UT-032	P5
FR-070	Prop distill	distillation	UT-071	P3
NFR-001	Lemma termination	nafs-core	PT-001	P1
NFR-002	Prop roofline; measured 94 tok/s prefill	nafs-core, backends	PT-002	P1
NFR-003	Measured 15.5 tok/s decode	backends	PT-003	P1
NFR-004	Measured 6.7 tok/s decode	backends	PT-004	P1
NFR-005	Prop roofline	backends, config	PT-005	P1
NFR-006	Prop roofline	nafs-memory, backends	UT-033	P1
NFR-007	Prop roofline	nafs-gateway	PT-006	P4
NFR-010	D-04 operations	nafs-gateway	IT-013	P2
NFR-011	IF-008	nafs-gateway	IT-014	P1
NFR-012	Draft risk: version drift	config, nafs-memory	UT-034	P0
NFR-020	Prop miscal; Prop samples	nafs-core, nafs-eval	ER-005	P2
NFR-021	D-04 operations	nafs-gateway	IT-015	P1
NFR-022	SR threat model	nafs-gateway	ST-002	P1
NFR-023	D-04 operations	workspace, CI	CI-001	P0

ID	Motivation	Component	Test	Phase
IF-001	D-03 interfaces	nafs-gateway	CT-001	P1
IF-002	FR-006	nafs-gateway	CT-002	P1
IF-003	FR-002	nafs-gateway	CT-003	P1
IF-004	FR-001	nafs-gateway	CT-004	P1
IF-005	D-03 interfaces	nafs-types	CT-005	P1
IF-006	D-02 stack	nafs-core (backend trait)	CT-006	P1
IF-007	D-04 operations	nafs-core	UT-035	P1
IF-008	Prop roofline	nafs-core	UT-036	P1
IF-009	FR-050, FR-052	nafs-gateway	CT-007	P4
IF-010	FR-013	nafs-memory	CT-008	P1
MR-001	D-02 roster	config	UT-037	P0
MR-002	D-02 stack	nafs-core	UT-038	P1
MR-003	A-004	deployment	IT-016	P0
MR-004	NFR-010	nafs-core	IT-017	P2
MR-005	H7	config	ER-011	P0
TR-001	Prop distill	distillation	UT-072	P3
TR-002	Prop distill (i)	distillation	UT-073	P3
TR-003	Prop distill (ii)	distillation	UT-074	P3
TR-004	Unsloth docs	distillation	IT-094	P3
TR-005	A-003	distillation	IT-094	P3
TR-006	NFR-012	distillation	UT-075	P3
TR-007	Prop distill (iii)	distillation, nafs-eval	ER-008	P3
TR-008	Prop distill (i)	distillation	UT-076	P3
SR-001	Threat model	nafs-core, nafs-memory	ST-003	P2
SR-002	Threat model	nafs-verify	ST-001	P1
SR-003	FR-034	nafs-verify	ST-004	P2
SR-004	Ethics	nafs-memory, distillation	ST-005	P3
SR-005	Sec. background rules	nafs-web, docs	RV-001	P6
SR-006	Threat model	deployment	ST-006	P5
OR-001	D-04 operations	workspace, CI	CI-001	P0
OR-002	NFR-012	deployment	CI-002	P5
OR-003	MR-003	deployment	IT-016	P5
OR-004	FR-042	deployment	IT-018	P5
OR-005	TR-007	deployment, config	IT-019	P3
OR-006	NFR-012	config	UT-039	P1
ER-001	Prop samples (i)	nafs-eval	ER-001	P0
ER-002	Prop samples (ii); Prop cascade	nafs-eval	ER-002	P2
ER-003	Thm precision	nafs-verify, nafs-eval	ER-003	P2
ER-004	Prop collision	nafs-eval	ER-004	P2
ER-005	Prop samples; draft defect 7	nafs-eval	ER-005	P2
ER-006	Lemma termination	workspace	CI-003	P1
ER-007	NFR-007	nafs-bench	PT-006	P4
ER-008	Draft risk: version drift	nafs-eval, CI	ER-008	P3
ER-010	Eval plan	nafs-eval	UT-092	P0
ER-011	H7	nafs-bench	ER-011	P0
NFR-030	D-07 CPU speed goal	nafs-core, nafs-gateway, nafs-web	UT-110	P7
NFR-031	A-044 ordering effect	nafs-bench, scripts	PT-010	P7

ID	Motivation	Component	Test	Phase
FR-080	Thm gate; D-07	nafs-core	UT-111	P7
FR-081	D-07; Phase-5 draft-model result	scripts, serving	PT-011	P7
ER-020	Lab mission	nafs-eval, nafs-web	ER-020	P7
FR-082	Thm 1; DTSR/Dynasor-CoT 2026; D-07	nafs-core	UT-112	P7

14 Open Issues and Risks

1. **Phase-0 estimates are placeholders.** $\hat{e}_2$, Δc, the Beta fit and the budget curve must be measured before the calibrated gate is enabled (FR-011, FR-021).
2. **Hybrid-model CPU performance.** Qwen3.5 on AVX2 has no controlled x86 benchmark in the literature; ER-011 resolves this.
3. **vLLM on the reference machine is untested** (requires Docker on Linux); the vLLM configuration is documented, not measured.
4. **Judge reliability.** The soft verifier shares weights with System 2; its (r, φ) may be poor on open domains, which would keep assurance at `consensus` for most non-verifiable replies.
5. **Budget forcing on vLLM** requires a re-issued completion with a partial reasoning prefix; the exact chat-template handling must be validated for each checkpoint.
6. **Slow-path latency on CPU** (minutes) limits interactive use; the GPU profile is the mitigation.
7. **Licence of LiveCodeBench data** and gating of GPQA must be confirmed before inclusion in the release suite.
8. **Unsloth version churn.** The training skeleton targets Unsloth 2026.10.x and transformers v5; pin both.
9. **Scholar review** of the analogy text (SR-005) is a release dependency with an external owner.